

Lokale KI für automatisierte Animationsvideos

Anleitung: Vom Prompt zum fertigen YouTube-Video – Text, Stimme und Bild komplett offline

1. Was wir eigentlich bauen wollen

Das Ziel: Du gibst einen **Prompt** ein (z. B. "Erkläre in 60 Sekunden, wie ein Vulkan entsteht"), und am Ende kommt automatisch ein fertiges Video heraus – mit Text, gesprochener Stimme und passenden Bildern oder Animationen. Das Ganze läuft komplett auf dem eigenen Rechner, ohne dass Daten an einen Cloud-Dienst geschickt werden.

Damit das funktioniert, brauchst du keine einzelne "Wunder-KI", sondern eine **Kette aus mehreren spezialisierten Modellen**, die nacheinander arbeiten – ähnlich wie in eurem Record-Studio-Projekt, wo auch mehrere Bausteine (Aufnahme, Szenen, Timeline) zusammenspielen.

Schritt	Aufgabe	Was übernimmt das?
1	Prompt verstehen & Skript schreiben	Lokales Sprachmodell (LLM)
2	Text in gesprochene Sprache umwandeln	Lokales Text-zu-Sprache-Modell (TTS)
3	Passende Bilder / Animation erzeugen	Lokales Bild-/Video-Diffusionsmodell
4	Alles zu einem Video zusammenfügen	ffmpeg / eigenes Python-Skript

2. Warum lokal und nicht über eine Cloud-API?

Cloud-APIs (wie z. B. Bild- oder Video-KIs von großen Anbietern) sind oft einfacher zu benutzen, haben aber Nachteile, die für dieses Projekt nicht funktionieren:

- **Datenschutz:** Skripte, Stimmen oder eigene Trainingsdaten verlassen nie den eigenen Rechner.
- **Keine laufenden Kosten:** Cloud-APIs berechnen meist pro Bild, pro Minute Sprache oder pro Video.
- **Volle Kontrolle:** Man kann Modelle anpassen, mit eigenem Stil trainieren oder Filter abschalten, was bei Cloud-Diensten meist nicht erlaubt ist.
- **Kein Internet nötig:** Sobald die Modelle heruntergeladen sind, läuft alles offline.

Nachteil: Lokale Modelle brauchen eine ordentliche Grafikkarte (GPU) und sind meist etwas langsamer oder weniger "poliert" als die großen Cloud-Modelle. Dafür gehört einem die ganze Pipeline.

3. Grundbegriffe: Was heißt eigentlich "Anlernen" und "Trainieren"?

Diese drei Begriffe werden oft durcheinandergeworfen. Für unser Projekt reicht es, den Unterschied zu kennen:

a) Fertig trainiertes Modell nutzen (Inferenz)

Man lädt ein bereits fertig trainiertes Modell herunter (z. B. ein Sprachmodell oder ein Bildmodell) und benutzt es einfach über einen Prompt. Es wird **nichts neu gelernt** – das Modell "weiß" schon alles, was es je wissen wird. Das ist der Normalfall und reicht für 90 % dieses Projekts völlig aus.

b) Prompting

Man bringt dem Modell nichts bei, sondern formuliert die Anfrage nur so genau, dass ein gutes Ergebnis herauskommt (z. B. "im Stil eines Comics, flaches Design, warme Farben"). Das ist keine Form von Training, sondern reine Bedienung des fertigen Modells.

c) Fine-Tuning (Nachtrainieren)

Man nimmt ein fertiges Modell und trainiert es mit **eigenen zusätzlichen Beispielen** weiter, damit es einen bestimmten Stil, eine bestimmte Stimme oder ein bestimmtes Thema besser beherrscht. Das ist deutlich aufwendiger und braucht eigene Trainingsdaten sowie Rechenzeit. Für den Einstieg ist Fine-Tuning **optional** – man kann damit später den eigenen Kanal-Stil festigen (siehe Kapitel 6).

d) LoRA (kleines, günstiges Fine-Tuning)

LoRA ist eine Methode, mit der man ein Modell nachtrainiert, ohne das komplette Modell neu zu berechnen. Es entsteht eine kleine Zusatzdatei (oft nur wenige MB), die dem Basismodell einen bestimmten Stil beibringt – z. B. "immer im Stil unserer eigenen Zeichnungen". Das ist der übliche Weg, wie Hobby-Projekte lokal eigene Stile trainieren.

4. Die Werkzeuge für jeden Schritt

Schritt 1 – Skript-Generierung mit einem lokalen LLM

Ein lokales Sprachmodell schreibt aus dem Prompt ein fertiges Sprecher-Skript, teilt es in Szenen ein und kann direkt auch Bildbeschreibungen für Schritt 3 mit ausgeben.

- **Ollama:** einfachste Art, Modelle wie Llama 3 oder Mistral lokal laufen zu lassen (auch per Kommandozeile oder über eine kleine lokale API, die dein Python-Programm ansprechen kann).
- **LM Studio:** Alternative mit grafischer Oberfläche, ebenfalls mit lokaler API.

Schritt 2 – Sprachausgabe (Text-to-Speech, TTS)

Wandelt den Skripttext in eine natürlich klingende Stimme um – wichtig für ein Erklär- oder Story-Video.

- **Piper:** sehr schnell, läuft auch auf schwächerer Hardware, viele Sprachen inkl. Deutsch.
- **Coqui TTS / XTTS-v2:** deutlich natürlicher klingend, kann sogar eine eigene Stimme aus wenigen Sekunden Beispielaudio nachbilden ("Voice Cloning") – das ist auch eine Form von Anlernen, nur sehr schnell und mit wenig Datenmaterial.

Schritt 3 – Bilder und Animation erzeugen

Hier entstehen die visuellen Inhalte, entweder als Einzelbilder pro Szene oder als kurze animierte Clips.

- **Stable Diffusion (über ComfyUI oder Automatic1111):** erzeugt Einzelbilder aus Textbeschreibungen, läuft komplett lokal, unterstützt auch eigene LoRA-Stile.
- **AnimateDiff / Stable Video Diffusion:** Erweiterungen, die aus Bildern kurze Bewegtbild-Clips machen.

- **Alternative für einfache Projekte:** Statt echter Videogenerierung reichen oft auch bewegte Standbilder (Zoom/Pan-Effekt, "Ken-Burns-Effekt") – deutlich schneller und mit weniger Hardware-Anspruch, und ein guter erster Schritt, bevor man echte Videomodelle ausprobiert.

Schritt 4 – Alles zusammensetzen

Ein Python-Skript (z. B. mit **moviepy** oder direkt mit **ffmpeg**) legt Bilder, Sprachspur und ggf. Untertitel übereinander und exportiert das fertige mp4 – genau wie es für den Video-Export in eurem Record-Studio-Projekt schon angedacht ist.

Idee für euer Projekt: Diese vier Schritte lassen sich später als weiterer Baustein in "Record Studio" einbauen – ein Button "KI-Szene generieren", der Skript, Stimme und Bild automatisch erzeugt und ins bestehende Timeline-System einfügt.

5. Was braucht der Rechner dafür?

Der größte Flaschenhals ist die Grafikkarte (GPU), speziell ihr Videospeicher (VRAM). Als grobe Orientierung:

Aufgabe	Minimum	Angenehm
Lokales LLM (7–8B Modelle)	8 GB VRAM oder CPU (langsam)	12–16 GB VRAM
TTS (Piper/Coqui)	läuft auch gut auf CPU	GPU beschleunigt nur zusätzlich
Bildgenerierung (Stable Diffusion)	6–8 GB VRAM	12+ GB VRAM
Videogenerierung (AnimateDiff/SVD)	12 GB VRAM	16–24 GB VRAM

Tipp: Fangt mit Schritt 1, 2 und dem "Ken-Burns-Effekt" aus Schritt 3 an. Das läuft auf fast jedem Rechner und liefert schon ein komplettes Video. Echte Videogenerierung ist der anspruchsvollste und optionale letzte Ausbaustufe.

6. Eigenen Stil trainieren (optional, für später)

Wenn der Grundworkflow läuft, kann man Schritt für Schritt eigene Trainingsdaten einbringen:

- 1 **Eigene Bildstil-LoRA:** 15–30 eigene oder ausgewählte Bilder im Wunschstil sammeln, damit ein LoRA für Stable Diffusion trainieren (Tools wie kohya_ss). Ergebnis: Alle generierten Bilder sehen wie "euer Kanal" aus.
- 2 **Eigene Sprecherstimme:** Mit XTTS-v2 reichen oft 10–30 Sekunden saubere Sprachaufnahme, um die Stimme nachzubilden – praktisch, wenn der Kanal immer dieselbe Erzählstimme haben soll.
- 3 **Eigener Skript-Stil:** Ein LLM lässt sich per Fine-Tuning oder auch nur über gute Beispiel-Prompts ("schreibe wie in diesen drei Beispielen") an einen bestimmten Erzählstil gewöhnen.

Für den Einstieg nicht nötig – das ist der Feinschliff, wenn die Basis-Pipeline schon zuverlässig läuft.

7. Zusammenfassung

- Es gibt kein einzelnes "Video-KI-Modell" – die Pipeline besteht aus mehreren spezialisierten, lokalen Modellen.
- Für den Start reicht "Prompting" mit fertigen Modellen völlig aus – Training/Fine-Tuning ist optional und kommt erst später für den eigenen Stil dazu.
- Reihenfolge zum Ausprobieren: 1) LLM für das Skript (Ollama) → 2) TTS für die Stimme (Piper) → 3) Bilder (Stable Diffusion) mit einfachem Zoom-Effekt → 4) Zusammenschnitt mit ffmpeg/moviepy.
- Alles läuft lokal, ohne Cloud-API und ohne laufende Kosten – nur die Grafikkarte setzt die Grenze für Tempo und Qualität.